

# آزمایشگاه سیستم عامل

## پروژه دوم : فراخوانی سیستمی

محمد امین رشید 810102454

سید محمد جواد شریفی 810102469

محمد صادق مهدوی پور 810102519

پرسش اول) در واقع ULIB نوعی متغیر است که شامل 4 تا فایل (ulib.c , usys.S , printf.c , umalloc.c) است که از آن ها به عنوان کتابخانه های سطح کاربر در برنامه هایمان استفاده میکنیم به طوری که در این فایل ها ، توابع مختلفی وجود دارد که در بعضی از آن ها فراخوانی سیستمی وجود دارد و همین موضوع باعث میشود به این توابع ، توابع wrapper بگوییم زیرا این توابع لایه ای بالاتر از توابع سیستم کال هستند و کاربر میتواند به این صورت از پیچیدگی های توابع سیستمی دور تر بماند و به صورت راحت تری از فراخوانی های سیستمی استفاده کند و درگیر جزییاتی مانند شماره سیستم کال ، آرگومان های سیستم کال ها ، return value ها و... نشود پس در 3 مورد این کار مزیت دارد:

1) ساده سازی توسعه برای برنامه نویس همانطور که توضیح دادم.مثلا در فایل ulib.c تابعی تحت عنوان stat وجود دارد که همانطور که از شکل زیر معلوم است، از 3 تا فراخوانی سیستمی مختلف استفاده کرده است:

```
70 int
71 stat(const char *n, struct stat *st)
72 {
73     int fd;
74     int r;
75
76     fd = open(n, O_RDONLY);
77     if(fd < 0)
78         return -1;
79     r = fstat(fd, st);
80     close(fd);
81     return r;
82 }
```

که ما در سطح user با فرخوانی تنها stat میتوانیم و دادن آرگومان های مناسب ، اطلاعات کامل یک فایل یا دایرکتوری رو به دست بیاوریم.

2) افزایش امنیت : با اضافه شدن این کتابخانه ، سطح دسترسی به سخت افزار از طریق سیستم کال مستقیم محدود میشود و میتوان با کنترل کردن این توابع wrapper این امنیت را برقرار نمود.

3) بهبود قابلیت حمل : از آنجایی که فرخوانی سیستمی کرنل های مختلف میتواند متفاوت باشد ، استفاده از فراخوانی های سیستمی از طریق این نوع کتابخانه ها میتواند این اختلاف را همدل کند به طوری که اگر اگر مثلا بخواهیم برنامه ای را از یک سیستم به یک سیستم دیگر منتقل کنیم ، کافی است از کتابخانه مربوطه آن کرنل استفاده کنیم . همین باعث میشود انتقال برنامه ها از xv6 به سیستم unix راحت تر شود.

تنها عیب این کار اضافه شدن سربار به فراخوانی های سیستمی باشد.

libc : این فایل شامل توابعی هست در سطح کاربر است و توابعی دارد که میتوان برای کار با رشته ها و سایر کار ها از آن استفاده کرد . توابعی همچون memmove , gets , stat , memset , strcpy و ....

printf.c , umalloc.c ها هم همچون libc شامل توابعی هستند که به ترتیب برای مدیریت حافظه و نوشتن استفاده می شوند و در برخی توابع آن ها فراخوانی سیستم مکرر انجام شده است.

usys.S یک کد اسمبلی است که به صورت زیر می باشد:

```
LAB2 > xv6 > rm usys.S
1  #include "syscall.h"
2  #include "traps.h"
3
4  #define SYSCALL(name) \
5      .globl name; \
6      name: \
7          movl $SYS_ ## name, %eax; \
8          int $T_SYSCALL; \
9          ret
10
11 SYSCALL(fork)
12 SYSCALL(exit)
13 SYSCALL(wait)
14 SYSCALL(pipe)
15 SYSCALL(read)
16 SYSCALL(write)
17 SYSCALL(close)
18 SYSCALL(kill)
19 SYSCALL(exec)
20 SYSCALL(open)
21 SYSCALL(mknod)
22 SYSCALL(unlink)
23 SYSCALL(fstat)
24 SYSCALL(link)
25 SYSCALL(mkdir)
26 SYSCALL(chdir)
27 SYSCALL(dup)
28 SYSCALL(getpid)
29 SYSCALL(sbrk)
30 SYSCALL(sleep)
31 SYSCALL(uptime)
```

این فایل شامل تعریف ماکرو است که این ماکرو رو برای تمامی سیستم کال ها استفاده شده است و تعریف ماکرو بدین صورت است که نام سیستم کال را به صورت یک name گلوبال تعریف میکند تا همه بتوانند استفاده کنند. همچنین شماره آن را در رجیستر eax ذخیره میکند تا کاربر نیاز نداشته باشد هنگام فراخوانی شماره

سیستم کال را بداند. (تمامی سیستم کال ها در `syscall.h` تعریف شده اند و در اینجا `include` شده است و هر کدام دارای یک شماره می باشد :

```
1 // System call numbers
2 #define SYS_fork 1
3 #define SYS_exit 2
4 #define SYS_wait 3
5 #define SYS_pipe 4
6 #define SYS_read 5
7 #define SYS_kill 6
8 #define SYS_exec 7
9 #define SYS_fstat 8
10 #define SYS_chdir 9
11 #define SYS_dup 10
12 #define SYS_getpid 11
13 #define SYS_sbrk 12
14 #define SYS_sleep 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23
```

`int $T_SYSCALL` هم برای دستور `interrupt` است ( برای سیستم کال نیاز به وقفه به کرنل داریم)  
و مقدار `$T_SYSCALL` برابر 64 است که کرنل متوجه شود این وقفه چه نوع وقفه ای است و با مراجعه به رجیستر `eax` متوجه شود که منظورمان کدام سیستم کال بوده است.  
این که این متغیر 64 است را از طریق فایل زیر میتوان متوجه شد:

```

LAB2 > xv6 > C traps.h > ...
7 // breakpoint
8 #define T_OFLOW 4 // overflow
9 #define T_BOUND 5 // bounds check
10 #define T_ILLOP 6 // illegal opcode
11 #define T_DEVICE 7 // device not available
12 #define T_DBLFLT 8 // double fault
13 // #define T_COPROC 9 // reserved (not used since
14 #define T_TSS 10 // invalid task switch segm
15 #define T_SEGNP 11 // segment not present
16 #define T_STACK 12 // stack exception
17 #define T_GPFLT 13 // general protection fault
18 #define T_PGFLT 14 // page fault
19 // #define T_RES 15 // reserved
20 #define T_FPERR 16 // floating point error
21 #define T_ALIGN 17 // alignment check
22 #define T_MCHK 18 // machine check
23 #define T_SIMDERR 19 // SIMD floating point error
24
25 // These are arbitrarily chosen, but with care not to overl
26 // processor defined exceptions or interrupt vectors.
27 #define T_SYSCALL 64 // system call
28 #define T_DEFAULT 500 // catchall
29
30 #define T_IRQ0 32 // IRQ 0 corresponds to int
31
32 #define IRQ_TIMER 0
33 #define IRQ_KBD 1
34 #define IRQ_COM1 4
35 #define IRQ_IDE 14
36 #define IRQ_ERROR 19
37 #define IRQ_SPURIOUS 31
38
39

```

پرسش دوم) مقایسه بین دستورات وقفه برای فراخوانی سیستمی:

(1) ابتدا دستور `int` را بررسی میکنیم :

یک دستوز قدیمی در اسمبلی است که نوعی وقفه نرم افزاری است که شماره وقفه را هم بعد از آن به کرنل میفرستد و باعث `interrupt` در سیستم عامل میشود. در این لحظه `OS` فرآیند اجرا را متوقف میکند و از طریق `table` به آدرس وقفه مربوطه میرود و سپس به سطح کرنل میرود و محتوای تمام رجیستر های `CPU` را در استک ذخیره میکند و سپس اکنون کرنل شماره سیستم کال مربوطه را از روی رجیستر مربوطه میخواند و سیستم کال را اجرا میکند .

دستور `sysenter/sysexit` در واقع روشی جدید برای وقفه دادن برای فراخوانی سیستمی است که در این مکانیزم از رجیستر های خاصی (`MSR`) به جای جدول وقفه استفاده میشود که 3 مورد را در این رجیستر ها ذخیره میکند : آدرس ورود به کرنل ، آدرس استک ، سگمنت کد کرنل.

به همین دلیل بدون جست و جو در جدول وقفه میتواند سریع به آدرس مشخص شده در کرنل برود و مود به کرنل تغییر میکند.

(2) به دلیل اینکه نیاز به جست و جو در جدول وقفه نبود و هم چنین از تعداد رجیستر های کمتری استفاده کردیم ، باعث میشود این روش بسیار سریع تر از روش `int` باشد و سربار سوییچ به کرنل کاهش می یابد.

(3) در اکثر `cpu` ها و حتی پردازنده های قدیمی میتوان از روش `int` استفاده کرد و برای پردازنده های قدیمی بهتر است از `int` استفاده شود زیرا پشتیبانی بهتری دارد و سازگاری بیشتری دارد.

ولی در سیستم های جدید تر مانند لینوکس 64 بیتی از همان روش های جدید تر به جای `int` استفاده میشود زیرا سربار سیستم کال را کاهش میدهد . هم چنین در سیستم های امبدد معمولا به دلیل سادگی پیاده سازی از `int` استفاده میشود .

پرسش سوم) در  $xv6$ ، فراخوانی‌های سیستمی با سطح دسترسی  $DPL\_USER$  فعال می‌شوند و با این سطح دسترسی، امکان فعال کردن سایر تله‌ها (یعنی تله‌های سخت‌افزاری و استثنایها) وجود ندارد. بنابراین اگر یک برنامه سطح کاربر بخواهد تله‌ای را فعال کند که سطح دسترسی آن تله، بیشتر از  $DPL\_USER$  باشد، سیستم‌عامل این اجازه را به برنامه نخواهد داد.

از علل این مکانیزم می‌توان به موارد زیر اشاره کرد:

۱- برای استفاده و فعال‌سازی یک گیت  $IDT$ ، باید سطح اولویت پردازش جاری ( $CPL$ )، کوچکتر یا مساوی سطح دسترسی توصیف‌گر تله ( $DPL$ ) باشد. بنابراین،  $CPL=3$  (حلقه کاربر)، فقط می‌تواند گیتی را صدا بزند که  $DPL=3$  داشته باشد. با توجه به اینکه تله‌های سطح هسته دارای  $DPL=0$  است، پس پردازش‌های سطح کاربر، امکان فعال‌سازی تله‌های سطح هسته را ندارند.

۲- اگر برنامه کاربر بتواند مستقیماً هر تله یا وقفه‌ای را فراخوانی کند، ممکن است به‌صورت ناخواسته (به‌دلیل باگ در کد یا اجرای نادرست یک دستور) وارد بخش‌هایی از هسته شود که برای آن شرایط آماده نیستند. چنین اتفاقی می‌تواند باعث خرابی وضعیت هسته، کرش سیستم یا از بین رفتن داده‌ها شود. بنابراین، نگه داشتن  $DPL=0$  برای استثنایها و وقفه‌ها باعث می‌شود که فقط خود پردازنده (در زمان وقوع واقعی آن رویداد) بتواند آن گیت‌ها را فعال کند، نه برنامه‌های کاربر.

۳- اگر کاربر بتواند هر تله‌ای را فعال کند، می‌تواند از آن برای دور زدن مکانیزم‌های امنیتی و کنترلی هسته استفاده کند. برای مثال، ممکن است با تحریک وقفه‌ها یا استثنایهای خاص، وارد مسیرهایی از کرنل شود که بررسی‌های امنیتی سیستم‌کال را ندارند. یا با ایجاد وقفه‌های پیاپی باعث از کار افتادن سیستم شود. (مباران تله -  $DoS$ )

پرسش چهارم) در معماری  $xv6$ ، هر پردازش دارای دو استک است. یک استک در فضای کاربر و دیگری در فضای هسته استفاده می‌شود. در این معماری، هرگاه یک تله یا وقفه رخ می‌دهد، سیستم بررسی می‌کند که آیا تغییر سطح دسترسی انجام می‌شود یا نه.

در صورتی که تغییر سطح دسترسی انجام شود، باید استک مورد استفاده نیز تغییر کند. به عنوان مثال اگر تغییر سطح دسترسی از فضای کاربر به فضای هسته انجام می‌شود، باید به جای استک کاربر از استک هسته استفاده شود. بنابراین در این شرایط، نیاز داریم که اطلاعات مورد نیاز برای بازیابی استک کاربر را در استک هسته ذخیره کنیم تا پس از رسیدگی به وقفه یا تله در فضای هسته، بتوانیم به اجرای پردازش کاربر ادامه دهیم. مقادیر  $ss$  و  $esp$  برای انجام این بازیابی مورد استفاده قرار می‌گیرند.  $ss$  مشخص می‌کند که استک کاربر در کدام قسمت از حافظه

ذخیره شده است و **eps**، آدرس دقیق سر استک را مشخص می‌کند. بنابراین با پوش کردن این دو اشاره‌گر در استک هسته، می‌توانیم پس از اجرای عملیات هسته، به حالت قبلی برگردیم و ادامه پردازش کاربر را اجرا کنیم. اما در صورتی که تغییر سطح دسترسی انجام نشود (مثلاً تله در همان سطح کرنل اتفاق بیفتد)، دیگر نیازی به بازیابی استک مربوط به وضعیت قبلی نداریم. چرا که پردازنده باید با همین پشته فعلی، عملیات را ادامه دهد. بنابراین در این حالت دیگر **ss** و **eps** را در استک پوش نمی‌کنیم.

پرسش پنجم) وظیفه این توابع این است که مقدار یا آدرس آرگومان‌ها را از استک مربوطه دریافت کنند و بررسی کنند که آیا این آدرس‌ها در محدوده مجاز حافظه **process** قرار داشته باشند و سپس اجازه استفاده از آنها را بدهند.

تابع **argint** مقدار اندیس آرگومانی که می‌خواهیم را می‌گیرد و سپس آدرس آرگومان با آن اندیس را محاسبه میکند. سپس بررسی میکند که آیا آدرس بدست آمده معتبر هست یا نه و اگر معتبر بود مقدار عدد صحیح را از آن آدرس می‌خواند و مقدار را در آرگومان دوم تابع ذخیره میکند.

تابع **argptr** برای دریافت آرگومان‌هایی از نوع پوینتر است که به قسمتی از حافظه اشاره میکند.

```
int argptr(int n, char **pp, int size);
```

اولین پارامتر همان شماره آرگومانی است که می‌خواهیم. آرگومان دوم پوینتری به یک پوینتر است که در اصل در آن آدرس حافظه مورد نظر ذخیره می‌شود. آرگومان سوم سایز بافر یا ... ای است که می‌خواهیم بخوانیم (البته در بعضی وقتها تعداد بایت‌هایی است که انتظار داریم از آن حافظه در دسترس باشند)

در صورت معتبر بودن آدرس، پوینتر به این آدرس در آرگومان دوم تابع ذخیره شده و **return 0** رخ میدهد در غیر اینصورت **-1 return**.

دلیل اینکه این تابع باید از **valid** بودن آدرس اطمینان حاصل کند این است که تنها در محدوده داده‌های **process** فعلی بتواند دسترسی داشته باشد و نتواند به داده‌های بقیه دسترسی پیدا کند که در این صورت

میتوان با درست کردن بدافزار در عملکرد برنامه های دیگر یا حتی هسته سیستم عامل مشکل ایجاد کرد و به داده ها دسترسی پیدا کرد.

**مثال عملی:** فرض کنید `read_sys(fd, buf, n)` بدون چک `buf` نوشته شود. کاربر می تواند `buf = آدرس` یا `NULL` یا آدرس یک صفحه آزاد شده یا آدرس حافظه هسته تنظیم کند؛ در نتیجه هسته روی آن آدرس می نویسد — یا `page fault` می دهد یا بدتر، حافظه حساس را بازنویسی می کند.

در نهایت هر چند با حذف چک کردن ها کارایی و سرعت سیستم عامل بیشتر خواهد شد اما بت توجه به توضیحات داده شده برای سیستم عامل مخاطره آمیز خواهد بود و این موضوع پذیرفته نیست.

میتوان بازه اشتباه وارد کرد اما در حالت فعلی سیستم به ما اجازه دسترسی به آن بازه را نخواهد داد.

پرسش ۶) تغییر دستی در رجیستر ها میتواند خیلی خطرناک باشد. رجیستر ها مسئولیت های متفاوتی دارند. از جمله نگه داری شماره سیستم کال. اگر رجیستر مورد نظر مقدارش تغییر کند سیستم کال مورد نظر عوض خواهد شد و نتیجه غیرقابل پیش بینی خواهد بود.

اگر رجیستر ها را طوری تغییر دهد که موقع خواندن آرگومان های مورد نظر به آدرس اشتباهی بخورد میتواند با خطا روبرو شود و یا داده اشتباهی را برای تابع مورد نظر بیاورد و باعث اشکال در روند اجرای برنامه و غیرقابل پیش بینی شدن خروجی شود.

اگر این آدرس برای توابعی که در حافظه مینویسند باشد میتواند باعث این شود که داده های `process` های دیگر تغییر کند.

به طور کلی این تغییر باعث تغییر عملکرد سیستم و یا حتی تغییر اطلاعات `process` های دیگر میشود که مطلوب نیست.



## - بررسی گام‌های اجرای فراخوانی سیستمی در سطح کرنل توسط gdb

در این قسمت در ابتدا یک برنامه سطح کاربر به نام `get_pid` می‌نویسیم که شماره پردازش فعلی را با استفاده از `getpid()` چاپ کند.

```
C get_pid.c > ...
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int main() {
6      int pid = getpid();
7      printf(1, "\npid = %d\n", pid);
8      exit();
9  }
```

سپس با استفاده از دستور `break syscall` یک breakpoint روی اولین دستور قابل اجرای تابع `syscall` می‌گذاریم. در ادامه برنامه `get_pid` را در شل `xv6` اجرا می‌کنیم تا در `gdb` به نقطه توقف برسیم.

حال دستور `bt` را اجرا می‌کنیم:

```
(gdb) bt
#0  syscall () at syscall.c:141
#1  0x8010787d in trap (tf=0x8dffefb4) at trap.c:43
#2  0x80107628 in alltraps () at trapasm.S:20
#3  0x8dffefb4 in ?? ()
Backtrace stopped: previous frame inner to this frame (corrupt stack?)
```

دستور `bt` (backtrace) پشته فراخوانی را از نقطه توقف فعلی تا قدیمی‌ترین فریم قابل دسترسی نشان می‌دهد. یعنی مشخص می‌کند که از فراخوانی کدام توابع به مکان فعلی رسیده‌ایم.

فریم 0: نشان می‌دهد که اجرای فعلی داخل تابع `syscall` متوقف شده است.

فریم 1: در این قسمت `trap` با آرگومان `tf=0x8dffefb4` فراخوانی شده است. این تابع، هندلر `trap` در کرنل است که می‌تواند با داشتن `tf`، تابع `syscall()` را برای اجرای یک درخواست فراخوانی کند. (مثلاً `getpid`)

فریم 2: تابع `alltraps` ورودی مشترک وقفه‌هاست. این تابع، رجیسترها را روی پشته ذخیره می‌کند و همچنین سگمنت‌های کرنل را تنظیم می‌کند تا پس از بازگشت بتوانیم به فضای کاربر برگردیم.

فریم 3: آدرسی است که `gdb` نتوانسته آن را به نماد یا سورسی متناظر کند. در نتیجه یک فریم ناشناخته است.

در این قسمت دستور down را اجرا می کنیم:

```
(gdb) down
Bottom (innermost) frame selected; you cannot go down.
```

هرگاه یک وقفه ایجاد می شود، gdb تمام فریم ها را در یک استک نشان می دهد. هرگاه بخواهیم از یک فریم به سمت فریم درونی تر (یعنی یک فراخوانی جدیدتر - دیرتر) برویم از down استفاده می کنیم و هر بار که دستور down را استفاده کنیم، می توانیم یک فریم پایین تر برویم.

(نکته: فریم، یک فراخوانی از یک تابع است. هر فریم شامل آدرس بازگشت، اشاره گر پشته، آرگومان ها و متغیرهای محلی همان فراخوانی مشخص است.)

اکنون باید مقدار رجیستر eax را چاپ کنیم:

```
(gdb) print myproc()->tf->eax
$1 = 5
```

همانطور که مشاهده می شود مقدار آن برابر ۵ است اما شماره فراخوانی سیستمی getpid() برابر ۱۱ می باشد. علت تفاوت این است وقتی یک breakpoint روی syscall قرار داده ایم، این یعنی، برنامه برای هر سیستم کالی می ایستد نه فقط برای getpid. در نتیجه وقتی برنامه شروع به اجرا می کند، در ابتدا روی فراخوانی های دیگری (مثل ... , read , write) می ایستد. بنابراین در شروع، مقدار tf->eax برابر شماره فراخوانی getpid() نیست. در ادامه با چند بار زدن c (ادامه دادن) به لحظه ای می رسیم که برنامه، getpid را صدا می زند. در این لحظه مقدار tf->eax برابر ۱۱ می شود.

```
(gdb) print myproc()->tf->eax
$12 = 11
```

همانطور که مشاهده می شود پس از ۱۲ بار اجرا، مقدار رجیستر eax با شماره فراخوانی getpid برابر شد. در انتها نیز خروجی برنامه get\_pid آورده شد است:

```
Booting from Hard Disk...
xv6...
cpu1: starting 1
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
Sadegh Mahdavi
javadi sharifi
Mohammad Amin Rashid
init: starting sh
$ get_pid
pid = 3
```

## ارسال آرگومان های فراخوانی های سیستمی:

تعاریف اولیه مربوطه را در فایل های زیر انجام میدهیم.

: User.h

```
int simple_arithmetic_syscall(int a, int b);
```

: Syscall.h

```
#define SYS_simple_arithmetic_syscall 22
```

: Syscall.c

```
extern int sys_simple_arithmetic_syscall(void);
```

```
[SYS_simple_arithmetic_syscall] sys_simple_arithmetic_syscall,
```

مرحله مهم این سیستم کال wrapper آن است. از ماکرو SYSCALL پیشفرض در `usys.s` نمیتوان استفاده کرد به این علت که آرگومان ها را در رجیستر های خواسته شده (رجیستر `ECX,EBX`) قرار نمیدهد. پس باید یک wrapper خودمان تنظیم کنیم:

```
.globl simple_arithmetic_syscall
simple_arithmetic_syscall:
    movl 4(%esp), %ebx
    movl 8(%esp), %ecx
    movl $SYS_simple_arithmetic_syscall, %eax
    int $T_SYSCALL
    ret
```

خط اول این تابع اسمبلی را گلوبال میکند که بتواند قابل فراخوانی توسط کد ها C باشد.

خط دوم نقطه شروع دستورات تابع است (`simple_arithmetic_syscall`)

خط سوم مقدار  $a$  را که به طور خودکار توسط کامپایلر زبان C در ۴ بایت ابتدایی  $esp$  قرار میگیرد را برداشته و در رجیستر  $EBX$  می ریزد.

خط چهارم مقدار  $b$  (آرگومان ورودی دوم) را که در ۸ بایت بالاتر پشته قرار دارد را به رجیستر  $ecx$  انتقال میدهد. خط پنجم شماره فراخوانی سیستمی (که پیشتر تعریف شد) را در رجیستر  $eax$  قرار میدهد.

این خط یک وقفه نرم افزاری برای اجرای سیستم کال است که باعث میشود پردازنده به حالت  $kernel mode$  سوییچ کند و تابع با شماره وارد شده درون  $eax$  را اجرا میکند.

خط آخر (نتیجه محاسبات در رجیستر  $EAX$  ریخته شده است) و به جایی که از آن کال شده بود باز میگردد.

حالا باید بدنه تابع را در قسمت سمت هسته تعریف کنیم که برای این کار سراغ فایل  $sysproc.c$  میرویم:

```
int sys_simple_arithmetic_syscall(void)
{
    int a, b, result;
    struct proc *curproc = myproc();

    a = curproc->tf->ebx;
    b = curproc->tf->ecx;

    result = (a - b) * (a + b);

    cprintf("Calc:  (%d - %d) * (%d + %d) = %d\n", a, b, a, b, result);

    return result;
}
```

در این تابع ابتدا یک پوینتر از نوع ساختار داده  $process$  تعریف شده که با کمک تابع  $myproc()$  به  $process$  در حال اجرا فعلی اشاره میکند. سپس از طریق  $trap frame$  آن  $process$  به رجیسترهای  $ebx$ ,  $ecx$  دسترسی پیدا میکنیم که مقدار  $a, b$  پیشتر در آنها ذخیره شده است. سپس مقدار محاسبه شده را پرینت میکنیم (طبق گفته سوال در سمت هسته پرینت میشود) و نتیجه هم  $return$  میشود.

در نهایت یک فایل test\_simple.c می سازیم که در آن محتوای زیر را قرار می دهیم:

```
int main(int argc, char *argv[])
{
    int a, b;

    if (argc != 3) {
        printf(2, "Usage: test_simple a b\n");
        exit();
    }

    a = atoi(argv[1]);
    b = atoi(argv[2]);

    int result = simple_arithmetic_syscall(a, b);

    exit();
}
```

که در آن ابتدا تعداد آرگومان ها چک میشود و اگر ۲ تا بود آنها را به int تبدیل کرده و تابع سیستم کال جدیدی را که درست کرده ایم را کال میکنیم و نتیجه را بدست می آوریم.

در نهایت برای استفاده از این تابع (کامند) در makefile در قسمت UPROGS مقدار \\_test\_simple را اضافه میکنیم.

مثال:

```
Sadegh Mahdavi
javad sharifi
Mohammad Amin Rashid
init: starting sh
$ test_simple 5 3
Calc: (5 - 3) * (5 + 3) = 16
$
```

که طبق مثال صورت پروژه نتیجه درست است.

## 1- پیاده سازی فراخوانی سیستمی ایجاد نسخه کپی فایل

برای اضافه کردن یک سیستم کال به صورت کلی باید در چند فایل تغییر ایجاد کنیم:

ابتدا به `user.h` تعریف تابع را اضافه میکنیم تا برنامه های سطح کاربر بتوانن تابع را صدا بزنند.

```
4 // system calls
5 int fork(void);
6 int exit(void) __attribute__((noreturn));
7 int wait(void);
8 int pipe(int*);
9 int write(int, const void*, int);
10 int read(int, void*, int);
11 int close(int);
12 int kill(int);
13 int exec(char*, char**);
14 int open(const char*, int);
15 int mknod(const char*, short, short);
16 int unlink(const char*);
17 int fstat(int fd, struct stat*);
18 int link(const char*, const char*);
19 int mkdir(const char*);
20 int chdir(const char*);
21 int dup(int);
22 int getpid(void);
23 char* sbrk(int);
24 int sleep(int);
25 int uptime(void);
26 int simple_arithmetic_syscall(int a, int b);
27
28 int make_duplicate(const char* src_fil);
29 int show_process_family(int pid);
30
```

سپس شماره ای جدید به سیستم کال تخصیص میدهیم که در فایل `syscall.h` این را اضافه میکنیم:

```
LAB2 > xv6 > C syscall.h > SYS_show_process_family
1 // System call numbers
2 #define SYS_fork 1
3 #define SYS_exit 2
4 #define SYS_wait 3
5 #define SYS_pipe 4
6 #define SYS_read 5
7 #define SYS_kill 6
8 #define SYS_exec 7
9 #define SYS_fstat 8
10 #define SYS_chdir 9
11 #define SYS_dup 10
12 #define SYS_getpid 11
13 #define SYS_sbrk 12
14 #define SYS_sleep 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23
24 #define SYS_make_duplicate 23
25
26 #define SYS_simple_arithmetic_syscall 22
27
28 #define SYS_show_process_family 24
```

سپس در `syscall` تعریف خود توابع سیستم کال درون کرنل را به صورت `extern` می نویسیم و همچنین به آرایه `syscalls` این سیستم کال را اضافه میکنیم تا از طریق شماره اش به این تابع درون کرنل بتوان نگاشت کرد.

```
5 extern int sys_uptime(void);
6 extern int sys_simple_arithmetic_syscall(void);
7
8 extern int sys_make_duplicate(void);
9 extern int sys_show_process_family(void);
10
11 static int (*syscalls[])(void) = {
12 [SYS_fork] sys_fork,
13 [SYS_exit] sys_exit,
14 [SYS_wait] sys_wait,
15 [SYS_pipe] sys_pipe,
16 [SYS_read] sys_read,
17 [SYS_kill] sys_kill,
18 [SYS_exec] sys_exec,
19 [SYS_fstat] sys_fstat,
20 [SYS_chdir] sys_chdir,
21 [SYS_dup] sys_dup,
22 [SYS_getpid] sys_getpid,
23 [SYS_sbrk] sys_sbrk,
24 [SYS_sleep] sys_sleep,
25 [SYS_uptime] sys_uptime,
26 [SYS_open] sys_open,
27 [SYS_write] sys_write,
28 [SYS_mknod] sys_mknod,
29 [SYS_unlink] sys_unlink,
30 [SYS_link] sys_link,
31 [SYS_mkdir] sys_mkdir,
32 [SYS_close] sys_close,
33
34 [SYS_make_duplicate] sys_make_duplicate,
35 [SYS_simple_arithmetic_syscall] sys_simple_arithmetic_syscall,
36 [SYS_show_process_family] sys_show_process_family
37 };
38
```

ایجاد `wrapper` : همانطور که در ابتدای این گزارش توضیح داده شد ، `wrapper` ها برای این است که کاربر فقط یک تابع رو صدا بزند و نیاز نداشته باشد بداند که شماره سیستم کال چند است و.....

برای همین به فایل `usys.S` از ماکرو `SYSCALL` استفاده میکنیم و نام آن تابعی که کاربر فرامیزد را درون آن قرار میدهیم ، کار این ماکرو این است که شماره این سیستم کال را در رجیستری ذخیره کند و سپس وقفه ای صدا میزند تا سیستم به حالت کرنل مود برود تا سیستم کال مربوطه در کرنل اجرا شود.

```

LAB2 > xv6 > asm usys.S
1  #include "syscall.h"
2  #include "traps.h"
3
4  #define SYSCALL(name) \
5      .globl name; \
6      name: \
7          movl $SYS_ ## name, %eax; \
8          int $T_SYSCALL; \
9          ret
10
11 SYSCALL(fork)
12 SYSCALL(exit)
13 SYSCALL(wait)
14 SYSCALL(pipe)
15 SYSCALL(read)
16 SYSCALL(write)
17 SYSCALL(close)
18 SYSCALL(kill)
19 SYSCALL(exec)
20 SYSCALL(open)
21 SYSCALL(mknod)
22 SYSCALL(unlink)
23 SYSCALL(fstat)
24 SYSCALL(link)
25 SYSCALL(mkdir)
26 SYSCALL(chdir)
27 SYSCALL(dup)
28 SYSCALL(getpid)
29 SYSCALL(sbrk)
30 SYSCALL(sleep)
31 SYSCALL(uptime)
32 SYSCALL(make_duplicate)
33 SYSCALL(show_process_family)
34

```

برای سایر سیستم کال همین روند باید طی شود.

اکنون میخواهیم سیستم کالی برای کپی کردن یک فایل ایجاد کنیم الان نیاز داریم که خود تابع سیستم کال را بنویسیم و به دلیل اینکه مربوط به کار با فایل است ، این تابع را در فایل **sysfile.c** اضافه میکنیم که برای سیستم کال های مربوط به کار با فایل است .

ابتدا در این تابع ، متغیر ها و پوینتر های مورد نیاز را تعریف میکنیم. **src name** که اسم فایل ورودی است را از طریق **argstr** میخوانیم که ورودی کاربر بوده و از سر استک میخوانیم. از استراکچر **inode** استفاده میکنیم که ساختاری برای هر فایل است که شامل فیلد هایی هست که اطلاعات مربوط به آن فایل را نشان میدهد.

همچنین با دو تا حلقه **while** اسم فایل جدید که **copy\_** به آن اضافه میشود را هندل میکنیم.



```

466
467 int sys_make_duplicate(void)
468 {
469     char *src_name;
470     char suffix[] = "_copy";
471     struct inode *ip_src;
472     struct inode *ip_dest;
473     char new_name[128];
474     int n;
475     char buf[512];
476     if (argstr(0, &src_name) < 0)
477         return -1;
478
479     int i = 0, j = 0;
480     while (src_name[i] != '\0')
481     {
482         new_name[i] = src_name[i];
483         i++;
484     }
485     while (suffix[j] != '\0')
486     {
487         new_name[i] = suffix[j];
488         i++;j++;
489     }
490     new_name[i] = '\0';
491     begin_op();
492     ip_src = namei(src_name);
493     if (!ip_src)
494     {
495         end_op();
496         return -1;
497     }
498     ilock(ip_src);
499
500     ip_dest = create(new_name, T_FILE, 0, 0);
501
502     if (!ip_dest)
503     {
504         iunlock(ip_src);
505         iput(ip_src);
506         end_op();
507         return 1;
508     }

```

سپس برای شروع عملیات فایل باید از `begin_op` استفاده کنیم تا ناسازگاری به وجود نیاید. این ناسازگاری مربوط به خواندن از دیسک ممکن است اتفاق بیفتد. سپس از طریق `namei`، ایندکس پوینتر فایل منبع را از طریق اسمش دریافت میکنیم. اگر چنین فایلی نبود -1 را بر میگردانیم. سپس فایل جدیدی را میسازیم از طریق `create` و `T_FILE` نشان میدهد یک فایل معمولی میخواهیم بسازیم نه مثلاً یک دایرکتوری.

سپس از طریق یک `while` به اندازه سائز بافر هر بار از `src` میخوانیم از طریق `readi` و از طریق سیستم کال `writel` آن را در `dest` مینویسیم. `n` تعداد بایت های خوانده شده است که هر بار `j` را به اندازه `n` جلو میبریم. و در آخر با سیستم کال هایی `ip_dest` رو آپدیت میکنیم و قفل فایل هارا باز میکنیم تا بقیه بتوانند دسترسی داشته باشند و از طریق `end_op` عملیات را پایان میدهیم.

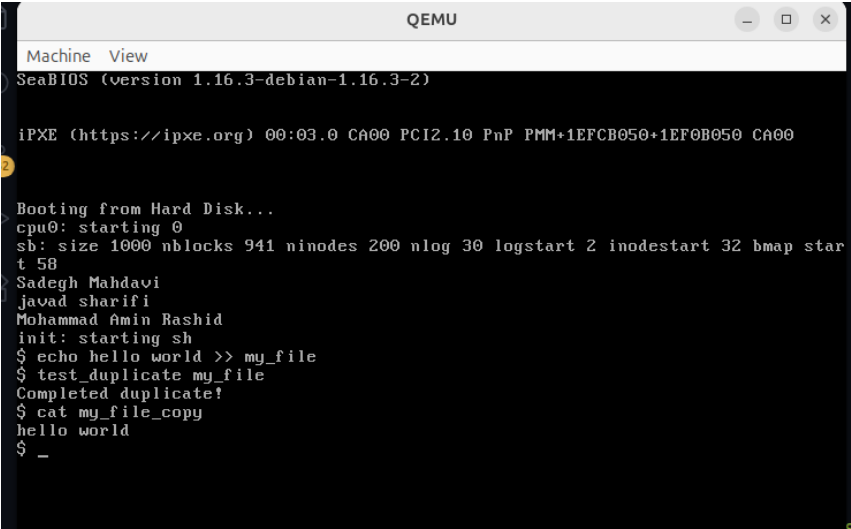
```
509
510
511     j = 0;
512     while (j < ip_src->size)
513     {
514         n = readi(ip_src, buf, j, sizeof(buf));
515         if (n <= 0){
516             break;
517         }
518         writel(ip_dest, buf, j, n);
519         j+= n;
520     }
521     ip_dest->size = ip_src->size;
522     iupdate(ip_dest);
523     iunlock(ip_src);
524     iunlock(ip_dest);
525     iput(ip_src);
526     iput(ip_dest);
527
528     end_op();
529
530     return 0;
531 }
532
533
534
```

تست: یک برنامه سطح کاربر به نام `test_duplicate` مینویسم و فراخوانی سیستمی را انجام میدهیم:

```

1  #include "types.h"
3  #include "user.h"
4
5  int main(int argc, char *argv[])
6  {
7
8      if (argc < 2)
9      {
10         printf(1, "\n");
11         exit();
12     }
13     char *src = argv[1];
14     int ret = make_duplicate(src);
15
16     if (ret == -1)
17     {
18         printf(1, "No file with this name!" );
19     }
20     else if (ret == 0)
21     {
22         printf(1, "Completed duplicate!" );
23     }
24
25     else
26     {
27         printf(1, "Error");
28     }
29
30     printf(1, "\n");
31     exit();
32 }

```



QEMU

Machine View

SeaBIOS (version 1.16.3-debian-1.16.3-2)

iPXE (https://ipxe.org) 00:03.0 CA00 PC12.10 PnP PMM+1EFCB050+1EF0B050 CA00

Booting from Hard Disk...

cpu0: starting 0

sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star

t 58

Sadegh Mahdavi

javad sharifi

Mohammad Amin Rashid

init: starting sh

\$ echo hello world >> my\_file

\$ test\_duplicate my\_file

Completed duplicate!

\$ cat my\_file\_copy

hello world

\$ \_

## 2- پیاده سازی فراخوانی سیستمی به دست آوردن اعضای خانواده یک پردازش

ابتدا مشابه قبل ، تغییرات لازم برای اضافه کردن یک سیستم کال جدید در فایل ها را اضافه میکنیم.

اینبار کد سیستم کال را در فایل `sysproc.c` می نویسیم ولی از آنجایی که `ptable` در فایل `proc.c` تعریف شده است در اینجا تابعی را فراخوانی میکنیم و بخش اصلی کد را در `proc.c` مینویسیم.

```
int sys_show_process_family(void)
{
    int pid;
    if(argint(0,&pid) < 0 ){
        return -2;
    }
    return show_process_family( pid);
}
```

ابتدا دو تا پوینتر به استراکت برای `p` و `target` در نظر میگیرم، زیرا میخواهیم لوپ بزنیم با `p` روی تمامی پراسس ها و در نهایت `p target` را پیدا کنیم.

لیست پراسس ها در `ptable` قرار دارد که یک استراکچر است که درونش آرایه ای از `process` ها را دارد.

توجه داریم که `pid` های پردازش ها از 1 شروع میشود و در ابتدا تمامی آن ها در آرایه مقدارشان صفر است و باید هنگام سرچ یک پردازش حواسمان باشد که `pid` صفر را رد کنیم. اگر چنین پردازش ای با `pid` داده شده توسط کاربر پیدا نشود منفی 1 را بر میگردانیم.

```

545
546 int show_process_family(int pid)
547 {
548     struct proc *p;
549     struct proc *target_p;
550     int found = 0;
551     int number_of_siblings = 0;
552     int number_of_children = 0;
553     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
554     {
555         if (pid == p->pid && p->pid != 0)
556         {
557             target_p = p;
558             found = 1;
559             break;
560         }
561     }
562
563     if (found == 0)
564     {
565         return -1;
566     }
567     int parent_id = target_p->parent->pid;
568     printf("My id: %d, My parent id: %d\n", pid, parent_id);

```

در ادامه دوتا حلقه میزنیم تا تعداد بچه ها و برادر های پردازنده را پیدا کنیم. این کار برای این است که اگر تعدادشان 0 باشد پیغام مناسبی بدهیم.

```

printf("My id: %d, My parent id: %d\n", pid, parent_id);
for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
{
    if (pid == p->parent->pid && p->pid != 0)
    {
        number_of_children += 1;
    }
}

for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
{
    if (p->pid != 0)
    {
        if (parent_id == p->parent->pid)
        {
            if (pid != p->pid)
            {
                number_of_siblings += 1;
            }
        }
    }
}

```

سپس با توجه به تعداد به دست آمده از قسمت قبل میتوانیم لیست آن ها را چاپ کنیم. برای پیدا کردن فرزندان یک پردازنده، روی تمامی پراسس ها لوپ میزنیم و چک میکنیم که آیا ایدی پدر همان ایدی پردازنده اصلی است یا نه.

زیرا در ساختار struct پردازش فقط parent نگه داری میشود.

برای siblings هم به همین صورت عمل میکنیم به صورت اینه چک میکنیم آیا پدر پردازش ها مشتک است و هم چنین پردازش های باید متمایز باشند.

```
if (number_of_children > 0)
{
    cprintf("Children of process %d:\n", pid);
    for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    {
        if (pid == p->parent->pid && p->pid != 0)
        {
            cprintf("Child pid: %d\n", p->pid);
        }
    }
}
else
{
    cprintf("This process does not have any children!\n");
}
if (number_of_siblings > 0)
{
    cprintf("Siblings of process %d:\n", pid);
    for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    {
        if (p->pid != 0)
        {
            if (parent_id == p->parent->pid)
            {
                if (pid != p->pid)
                {
                    cprintf("Sibling pid: %d\n", p->pid);
                }
            }
        }
    }
}
else
{
    cprintf("This process does not have any siblings!\n");
}

return 0;
}
```

تست با برنامه سطح کاربر:

LAB2 > xv6 > C show\_process\_family.c > main(int, char \* [])

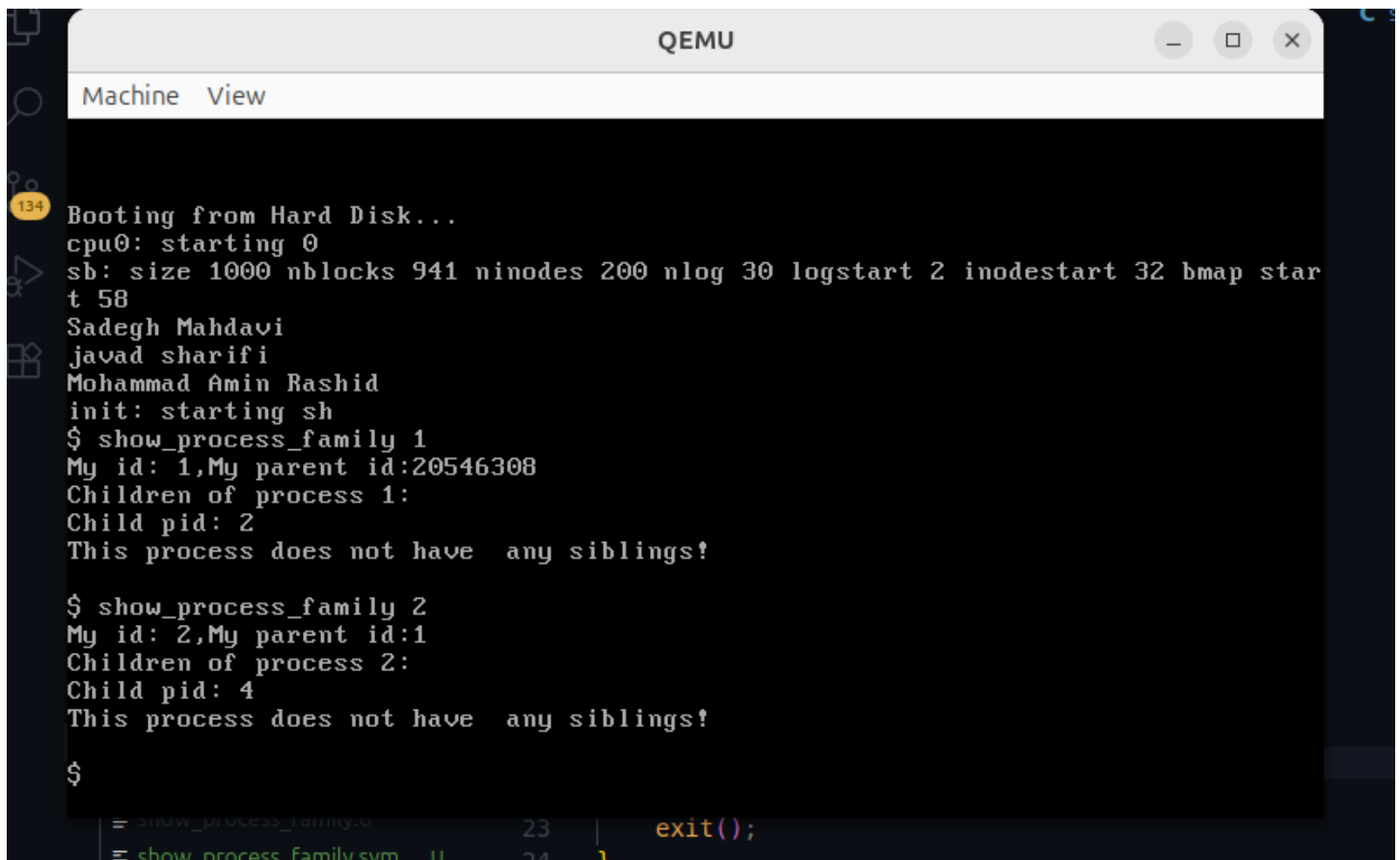
```
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int main(int argc, char *argv[])
6  {
7
8      if (argc < 2)
9      {
10         printf(1, "\n");
11         exit();
12     }
13     int pid = atoi(argv[1]);
14     int ret = show_process_family(pid);
15
16     if (ret == -1)
17     {
18         printf(1, "Not found process" );
19     }
20
21
22     printf(1, "\n");
23     exit();
24 }
```

همانطور که میدانیم در xv6 وقتی شروع به کار میکند ۲ تا پردازش در حال اجرا می باشد:

اولی init با pid=1 و دومی sh (shell) با pid=2 .

در برنامه زیر این 2 مورد را چک میکنیم و pid=1 ریشه است و پدر ندارد برای همین pid پدرش یک داده رندوم از حافظه است.

شماره pid برای پردازش روندی صعودی دارد. یعنی بعد از اجرای دستور ، بچه sh یک پردازش با ایدی 4 میشود و اگر دوباره از شل استفاده کنیم ، این پردازش با ایدی 4 terminate میشود و بعدی 5 است.



```
QEMU
Machine View
134 Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
Sadegh Mahdavi
javad sharifi
Mohammad Amin Rashid
init: starting sh
$ show_process_family 1
My id: 1,My parent id:20546308
Children of process 1:
Child pid: 2
This process does not have any siblings!

$ show_process_family 2
My id: 2,My parent id:1
Children of process 2:
Child pid: 4
This process does not have any siblings!

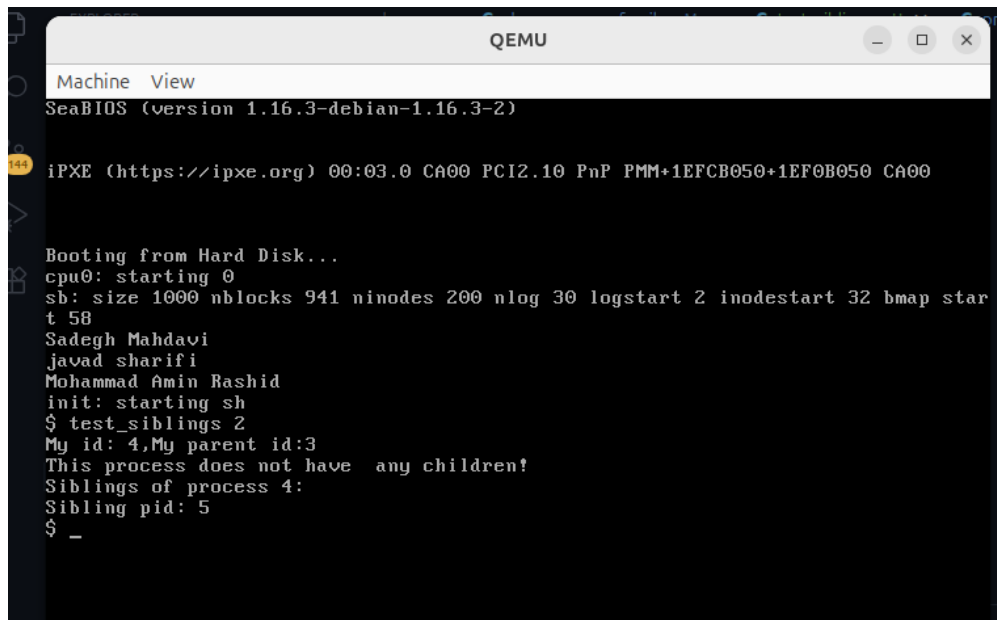
$
= show_process_family.o 23 | exit();
= show_process_family.sym 24 |
```

البته برای تست کردن siblings نیاز داریم برنامه ای بنویسیم که چند تا fork انجام دهد و سپس همین سیستم کال رو برای یکی از فرزندان اجرا کند که نتیجه در صفحه بعد می بینیم:

در این کد 2 تا فرزند fork میکند و سپس show\_process\_id را برای فرزند اول صدا میزنیم با این دستور مطمئن میشویم که بخش siblings هم به درستی کار میکند .



```
32 > xv6 > C test_siblings.c > main(int, char * [])
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int main(int argc, char *argv[])
6  {
7      int n = 3;
8      if (argc == 2)
9      {
10         n = atoi(argv[1]);
11     }
12     int pids[3];
13     for (int i = 0; i < n; i++)
14     {
15         int pid = fork();
16         if (pid == 0)
17         {
18             sleep(200);
19             exit();
20         }
21         else
22         {
23             pids[i] = pid;
24         }
25     }
26     int ret = show_process_family(pids[0]);
27     for (int i = 0; i < n; i++)
28     {
29         wait();
30     }
31     exit();
32 }
```



```
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFCB050+1EF0B050 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
Sadegh Mahdavi
javad sharifi
Mohammad Amin Rashid
init: starting sh
$ test_siblings 2
My id: 4, My parent id: 3
This process does not have any children!
Siblings of process 4:
Sibling pid: 5
$ _
```

### ۳- پیاده‌سازی فراخوانی سیستمی جستجوی محتوا در فایل

برای اضافه کردن این سیستم‌کال، ابتدا تمام تغییرات لازم را که برای اضافه کردن هر سیستم‌کالی باید انجام دهیم را اعمال می‌کنیم. (اضافه کردن سیستم‌کال به فایل‌های `syscall.h`، `user.h`، `usys.s` و `syscall.c`)

سپس از آنجا که این فراخوانی مربوط به کار با فایل است آن را در فایل `sysfile.c` تعریف می‌کنیم:

در ابتدا یک تابع کمکی می‌نویسیم که وظیفه آن جستجوی یک زیررشته در یک رشته است. در صورتی که زیررشته را در رشته پیدا کند، ۱ و در غیر اینصورت -۱ برمیگرداند:

```
int find_substr(const char *str, int str_len, const char *substr, int substr_len)
{
    if(substr_len <= 0 || str_len < substr_len)
        return -1;

    for(int i = 0; i + substr_len <= str_len; i++){
        int j = 0;
        while(j < substr_len && str[i + j] == substr[j])
            j++;
        if(j == substr_len)
            return 1;
    }

    return -1;
}
```

حال تابع اصلی را می‌نویسیم. در بخش اول کد، آرگومان‌های سیستم‌کال را دریافت می‌کنیم و سپس طول keyword را بدست می‌آوریم تا بعداً بتوانیم به تابع `find_substr` بدهیم:

```

int sys_grep_syscall(void)
{
    char *keyword = 0;
    char *filename = 0;
    char *user_buffer = 0;
    int buffer_size = 0;

    if(argstr(0, &keyword) < 0)
        return -1;

    if(argstr(1, &filename) < 0)
        return -1;

    if(argptr(2, &user_buffer, 0) < 0)
        return -1;

    if(argint(3, &buffer_size) < 0)
        return -1;

    int klen = 0;
    while(keyword[klen] != 0){
        klen++;
    }
    if(klen == 0)
        return -1;
}

```

در ادامه، یک اشاره‌گر به فایل تعریف می‌کنیم. (هر فایل روی دیسک یک inode دارد که شامل اطلاعات آن فایل است.) به کمک تابع namei، از روی نام فایل، inode آن را در دیسک پیدا می‌کنیم. سپس فایل را قفل می‌کنیم تا در حین خواندن، فرایند دیگری inode را تغییر ندهد.

سپس دو بخش از حافظه هسته را allocate می‌کنیم. یکی برای ذخیره فایل (kernel\_buf) و دیگری برای ذخیره هر خط فایل (kernel\_line):

```

int return_value = -1;
struct inode *ip = 0;

begin_op();
ip = namei(filename);
if(ip == 0){
    end_op();
    return -1;
}

ilock(ip);

int chunk = BSIZE;
char *kernel_buf = (char*)kalloc();
char *kernel_line = (char*)kalloc();
if(kernel_buf == 0 || kernel_line == 0){
    if(kernel_buf) kfree(kernel_buf);
    if(kernel_line) kfree(kernel_line);
    iunlockput(ip);
    end_op();
    return -1;
}

```

در ادامه، عملیات اصلی تابع در یک حلقه `while` انجام می‌شود. در هر مرحله یک `chunk` (بلوک) از فایل را خوانده و در `kernel_buf` ذخیره می‌کنیم. سپس تا جایی که به کاراکتر `"\n"` برسیم، کاراکترها را در `kernel_line` ذخیره می‌کنیم. سپس وجود `keyword` در `kernel_line` را به کمک تابع `find_substr` بررسی می‌کنیم. در صورتی که وجود داشت، به کمک تابع `copyout`، خط یافت شده را به فضای کاربر منتقل می‌کنیم.

در ادامه بررسی می‌کنیم که اگر بافر هنوز جای خالی داشت (`copy_len < buffer_size`) یک کاراکتر `"\0"` در انتهای رشته بگذاریم تا رشته به درستی پایان یابد.

در آخر نیز مقدار بازگشتی را برابر `copy_len` قرار می‌دهیم و دستورات انتهایی (`done`) را انجام می‌دهیم:

```
int line_len = 0;
int curr_pos = 0;
int read_len;

while((read_len = readi(ip, kernel_buf, curr_pos, chunk)) > 0){
    int i = 0;
    while(i < read_len){
        char c = kernel_buf[i++];

        if(c != '\n'){
            kernel_line[line_len++] = c;
            continue;
        }

        if(find_substr(kernel_line, line_len, keyword, klen) == 1){
            int copy_len = line_len;
            if(copy_len > buffer_size)
                copy_len = buffer_size;
            if(copyout(myproc()->pgdir, (uint)user_buffer, kernel_line, copy_len) < 0){
                return_value = -1;
            } else {
                if(copy_len < buffer_size){
                    char nul = 0;
                    copyout(myproc()->pgdir, (uint)user_buffer + copy_len, &nul, 1);
                }
                return_value = copy_len;
            }
            goto done;
        }

        line_len = 0;
    }
    curr_pos += read_len;
}
```

تنها مشکلی که ممکن است با آن مواجه شویم این است که اگر خط آخر فایل `"\n"` نداشته باشد، پردازش نمی‌شود. برای حل این مشکل، پس از پایان حلقه، در صورتی که خط آخر پردازش نشده باشد (`line_len > 0`) آن را پردازش می‌کنیم:

```

if (line_len > 0) {
    if (find_substr(kernel_line, line_len, keyword, klen) == 1) {
        int copy_len = line_len;
        if (copy_len > buffer_size)
            copy_len = buffer_size;

        if (copyout(myproc()->pgdir, (uint)user_buffer, kernel_line, copy_len) < 0) {
            return_value = -1;
        } else {
            if (copy_len < buffer_size) {
                char nul = 0;
                copyout(myproc()->pgdir, (uint)user_buffer + copy_len, &nul, 1);
            }
            return_value = copy_len;
        }
        goto done;
    }
}
}

```

در پایان نیز در صورتی که keyword را در هیچ یک از خطوط پیدا نکنیم، مقدار بازگشتی را برابر -۱ می‌گذاریم و حافظه‌هایی که allocate کرده بودیم را آزاد کرده، قفل فایل را برمی‌داریم و کار با فایل را خاتمه می‌دهیم:

```

return_value = -1;

done:
    kfree(kernel_buf);
    kfree(kernel_line);
    iunlockput(ip);
    end_op();
    return return_value;
}

```

برای تست کردن این سیستم‌کال، یک برنامه سطح کاربر به نام `grep_test` می‌نویسیم که این سیستم‌کال را فراخوانی کرده و نتیجه را چاپ می‌کند. همچنین این برنامه را در `UPROGS` در `Makefile` اضافه می‌کنیم:

```

C grep_test.c > main(int, char *[])
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  #define BUFSZ 512
6
7  int main(int argc, char *argv[])
8  {
9
10     printf(1, "\n");
11
12     if (argc != 3) {
13         printf(1, "\n");
14         exit();
15     }
16
17     char buf[BUFSZ];
18     int n = grep_syscall(argv[1], argv[2], buf, BUFSZ);
19
20     if (n < 0) {
21         printf(1, "not found or error\n");
22         exit();
23     }
24
25     printf(1, "%s\n", buf);
26
27     exit();
28 }

```

در پایان نیز برای صحت‌سنجی عملکرد این سیستم‌کال، ۴ تست انجام شده است که دو مورد اول آن مربوط به حالت موفقیت و دو مورد بعدی مربوط به حالت شکست است. (یکی زمانی که keyword یافت نشد و دیگری زمانی که file یافت نشد):

```
Booting from Hard Disk...
cpu1: starting 1
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
Sadegh Mahdavi
javad sharifi
Mohammad Amin Rashid
init: starting sh
$ grep_test spi README
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
$ grep_test lovi README
Wolovick, wxdao, Grant Wu, Jindong Zhang, Icenowy Zheng, and Zou Chang Wei.
$ grep_test sssss README
not found or error
$ grep_test is file.txt
not found or error
$
```

#### ۴- پیاده‌سازی فراخوانی سیستمی تنظیم اولویت:

ابتدا مانند بقیه سیستم کال های ایجاد شده تغییرات را در فایل های syscall.h, user.h, usys.S, syscall.c تغییرات لازم را انجام می‌دهیم (به ترتیب):

```
#define SYS_set_priority_syscall 26
```

```
int set_priority_syscall(int pid, int priority);
```

```
SYSCALL(set_priority_syscall)
```

```
extern int sys_set_priority_syscall(void);
```

```
[SYS_set_priority_syscall] sys_set_priority_syscall,
```

از مهمترین تغییرات این است که در فایل proc.h جایی که استراکچر proc تعریف شده است مقداری برای اولویت برای آن در نظر بگیریم:



```

struct proc {
    uint sz;
    pde_t* pgdir;
    char *kstack;
    enum procstate state;
    int pid;
    struct proc *parent;
    struct trapframe *tf;
    struct context *context;
    void *chan;
    int killed;
    struct file *ofile[NOFILE];
    struct inode *cwd;
    char name[16];
    int priority;
};

```

مقدار اولویت ها را ۳ نوع در نظر میگیریم. ۰ برای اولویت بالا ۱ عادی و ۲ پایین.

باید حواسمان باشد که زمانی که یک process در حال ایجاد است یک مقدار default برای آن در نظر بگیریم. پس به فایل proc.c رفته و در تابع allocproc این خط را اضافه میکنیم:

```

found:
    p->state = EMBRYO;
    p->pid = nextpid++;
    p->priority = 1;

```

که برای آن اولویت ۱ (عادی) در نظر گرفته ایم.

حالا باید تابعی کمکی نوشته شود که در صورتی که این سیستم کال با دو آرگومان PID, priority فراخوانی شد process را پیدا کرده و مقدار اولویت آن را آپدیت کند و اگر process ای با PID ورودی وجود نداشت مقدار ۱- را برگرداند و در صورت موفقیت ۰ را برگرداند (طبق صورت پروژه)

حالا تابع sys\_set\_priority\_syscall را به صورت زیر تعریف میکنیم (در فایل sysproc.c):

```

int sys_set_priority_syscall(void)
{
    int pid;
    int priority;

    if(argint(0, &pid) < 0)
        return -1;
    if(argint(1, &priority) < 0)
        return -1;

    if(priority < 0 || priority > 2) {
        cprintf("Error: Priority must be between 0 (High) and 2 (Low).\n");
        return -1;
    }

    return set_priority_helper(pid, priority);
}

```

به این صورت کار میکند که آرگومان اول را از پشته این تابع میخواند و در `pid` میریزد و آرگومان دوم را در `priority` می ریزد و اگر هر کدام نبود -۱ را برمیگرداند و اگر در نهایت مقدار اولویت بین ۰ تا ۲ نبود (مقدار قانونی نبود) ارور پرینت میکند و -۱ برمیگرداند. در غیر اینصورت تابع کمکی که بالاتر توضیح داده شد را کال کرده و مقدار خروجی آن را برمیگرداند.

تا به اینجا مقدار اولویت هر `process` را تنظیم کرده ایم. حالا مهمترین بخش کار این است که تابعی که وظیفه مدیریت اجرای پراسس ها و زمان بندی آنها را در اختیار دارد را طوری تغییر دهیم که طبق خواسته ما (طبق اولویت بالاتر) `process` ها را اجرا کند.

(برای اینکه بی نیاز به تعریف `prototype` تابع کمکی در هر فایل مورد نیاز باشیم آن را در فایل `defs.h` تعریف میکنیم:

```

int set_priority_helper(int pid, int priority);

```

حالا همانطور که توضیح داده شد باید تابع `scheduler` در فایل `proc.c` که وظیفه زمان بندی اجرای `process` ها را دارد را تغییر دهیم.

تابع باید به این صورت کار کند که ابتدا در همه process ها لوپ بزند و اگر با اولویت بالاتر چیزی بود اول آن را اجرا کند و اگر همه اولویت های ۰ تمام شدند سپس به اولویت ۱ برود و در صورت تمام شدن همه آنها به اجرای process ها با اولویت ۲ برود. یک فلگ برای پیدا شدن process با اولویت صفر تعریف میکنیم:

```
found_runnable_at_level_zero = 0;
```

در لوپ زیر بین process های آماده اجرا در صورتی که process ای با اولویت ۰ پیدا شد آن را به حالت در حال اجرا در میآورد و فلگ را ۱ میکند. و سپس مانند کد قبلی عمل میکند.

```
for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
    if (p->state == RUNNABLE && p->priority == 0) {
        found_runnable_at_level_zero = 1;

        // Switch to chosen process. It is the process's job
        // to release ptable.lock and then reacquire it
        // before jumping back to us.
        c->proc = p;
        switchvm(p);
        p->state = RUNNING;

        swtch(&(c->scheduler), p->context);
        switchkvm();

        // Process is done running for now.
        c->proc = 0;
    }
}
```

برای حالتی که یک process با استتیت آماده اجرا با اولویت ۰ وجود نداشته باشد گذر میکنیم و به لوپ بعدی میرویم ولی در صورتی که وجود داشت قفل را برداشته و ادامه میدهیم (برای پیدا کردن process احتمالی بعدی با همین شرایط)

```
if (found_runnable_at_level_zero) {
    release(&ptable.lock);
    continue;
}
```

همین کار را برای اولویت با شماره ۱ هم انجام میدهیم:

```

int found_runnable_at_level_one = 0;
for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
    if (p->state == RUNNABLE && p->priority == 1) {
        found_runnable_at_level_one = 1;

        c->proc = p;
        switchvm(p);
        p->state = RUNNING;

        swtch(&(c->scheduler), p->context);
        switchkvm();

        c->proc = 0;
    }
}

if (found_runnable_at_level_one) {
    release(&ptable.lock);
    continue;
}

```

اگر در کل process ها چیزی با شرایط گفته شده (با اولویت ۱) پیدا نشد برای اولویت آخر (۲) سرچ میکنیم:

```

for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
    if (p->state == RUNNABLE && p->priority == 2) {
        c->proc = p;
        switchvm(p);
        p->state = RUNNING;

        swtch(&(c->scheduler), p->context);
        switchkvm();

        c->proc = 0;
    }
}

release(&ptable.lock);

```

و در نهایت چون یا پیدا میشود و باید به بالا برگردیم یا process های آماده به اجرا تمام شده اند و چیزی پیدا نمی شود قفل را برمیداریم.

حالا همه چیز به درستی پیاده سازی شده است و فقط کافیست آن را مطابق خواسته 'تست کنیم.

برنامه تست test\_priority.c :

یک برنامه سنگین درست کرده ایم که ۱ میلیارد بار لوپ میزند:

```
void heavy_loop(char *process_type)
{
    long long i;
    long int count = 1;
    printf(1, "Process %s: STARTED\n", process_type);

    for (i = 0; i < 1000000000; i++)
        count += 1;

    printf(1, "Process %s: FINISHED\n", process_type);
}
```

حالا باید دو تا process ایجاد کنیم که هر دو این برنامه را اجرا کنند یکی با اولویت بالاتر و دیگری با اولویت پایین تر:

```

int main(int argc, char *argv[])
{
    int pid_low, pid_high;

    printf(1, "Parent: priority test started\n");

    pid_low = fork();
    if (pid_low == 0) {
        heavy_loop("LowPriority");
        exit();
    }

    pid_high = fork();
    if (pid_high == 0) {
        heavy_loop("HighPriority");
        exit();
    }

    if (pid_low > 0 && pid_high > 0) {
        set_priority_syscall(pid_low, 2);
        set_priority_syscall(pid_high, 0);
        wait();
        wait();
        printf(1, "Parent: both children finished. test completed.\n");
    }

    exit();
}

```

این برنامه با استفاده از `fork()` دو تا `child` ایجاد میکند که هر کدام تابع سنگین را باید اجرا کنند (در نهایت هم `exit()` گذاشته ام که بقیه برنامه را `child` ها اجرا نکنند). سپس `process` والد به یکی اولویت ۲ و دیگری ۰ را میدهد (با داشتن PID آنها) سپس منتظر تمام شدن هر دو برنامه میشود. خروجی به صورت زیر خواهد بود:

```
Sadegh Mahdavi
javad sharifi
Mohammad Amin Rashid
init: starting sh
$ test_priority
Parent: priority test started
Process HighPriority: STARTED
Process HighPriority: FINISHED
Process LowPriority: STARTED
Process LowPriority: FINISHED
Parent: both children finished. test completed.
^
```

که واضحا نشان میدهد با اینکه process با اولویت بالاتر بعد از process با اولویت پایین تر تعریف شد در کد ما اما process با اولویت بالاتر زودتر به اتمام رسیده است پس پیاده سازی ما درست بوده است.